A PROJECT REPORT

on

“AI SHOPPING AGENT”

Submitted to

KIIT Deemed to be University

In Partial Fulfilment of the Requirement for the Award of

BACHELOR’S DEGREE IN
INFORMATION TECHNOLOGY

BY

Ridhi Kumari

23052632

UNDER THE GUIDANCE OF

GUIDE NAME

SCHOOL OF COMPUTER ENGINEERING

KALINGA INSTITUTE OF INDUSTRIAL TECHNOLOGY

BHUBANESWAR, ODISHA - 751024

August 2026

A PROJECT REPORT

on

“AI SHOPPING AGENT”

Submitted to

KIIT Deemed to be University

In Partial Fulfilment of the Requirement for the Award of

BACHELOR’S DEGREE IN
INFORMATION TECHNOLOGY

BY

Ridhi Kumari

23052632

UNDER THE GUIDANCE OF

GUIDE NAME

SCHOOL OF COMPUTER ENGINEERING

KALINGA INSTITUTE OF INDUSTRIAL TECHNOLOGY

BHUBANESWAR, ODISHA - 751024

August 2026

CERTIFICATE

This is to certify that the project entitled

“AI SHOPPING AGENT“

submitted by

Ridhi Kumari

23052632

is a record of bona fide work carried out by the student, in partial fulfilment of the requirement for the award of Degree of Bachelor of Engineering (Computer Science & Engineering OR Information Technology) at KIIT Deemed to be University, Bhubaneswar. This work was completed during the year 2025-2026, under the guidance of the Project Guide.

Date://

(Guide Name)
Project Guide

Signature of Ridhi Kumari

Acknowledgements

I am profoundly grateful to GUIDE NAME of School of Computer Engineering, KIIT University for their guidance and encouragement throughout the development of AI Shopping Agent. I also acknowledge the open-source communities behind Shopify developer tools, Next.js, React, Vercel AI SDK, PostgreSQL, Redis, and Playwright, whose documentation and software supported this work. .....

Signature of Ridhi Kumari

ABSTRACT

AI Shopping Agent is a web-based conversational commerce system that converts natural-language buying intent into grounded product recommendations. The system asks a clarifying question only when a missing constraint would materially change the result set, searches the live Shopify catalog across merchants, compares relevant products and sellers, and presents direct merchant purchase links through structured interface components.

The application combines a Next.js and React frontend, a TypeScript agent backend, Shopify Catalog MCP, Vercel AI SDK streaming, OpenRouter or DeepSeek model access, PostgreSQL persistence, and Redis caching for non-catalog support data. Grounding is enforced through tool-derived product cards and comparison views rather than model-generated product tables. Backend unit tests, browser-oriented end-to-end tests, and an eight-scenario agent self-test harness support verification.

Keywords: Agentic commerce, Shopify Catalog MCP, generative UI, product recommendation, tool calling, Next.js, conversational shopping.

Contents

List of Figures

Chapter 1

Introduction

Online product discovery remains difficult when a buyer's need is expressed as intent rather than as an exact product name. Conventional keyword search returns long result lists, forces the buyer to translate requirements into filters, and makes cross-merchant price and seller comparison tedious. Fast-moving categories add another risk: a technically matching result may be old, poorly rated, unavailable, or unrelated to the requested product class.

AI Shopping Agent addresses this gap with a conversational workflow grounded in Shopify Catalog MCP. The agent clarifies only decisive missing constraints, searches live catalog data, refines or paginates results, compares two to four products, compares sellers, verifies research claims through web evidence, and emits a checkout call to action only after the buyer chooses an item. Structured product cards and comparison components are generated from tool outputs, reducing the opportunity for fabricated prices, sellers, specifications, or links.

The report first explains the relevant technical foundations, then defines requirements and architecture, presents implementation and verification, records the standards followed, and closes with limitations and future scope. Figure 1.1 shows the implemented chat surface used to begin a shopping conversation.

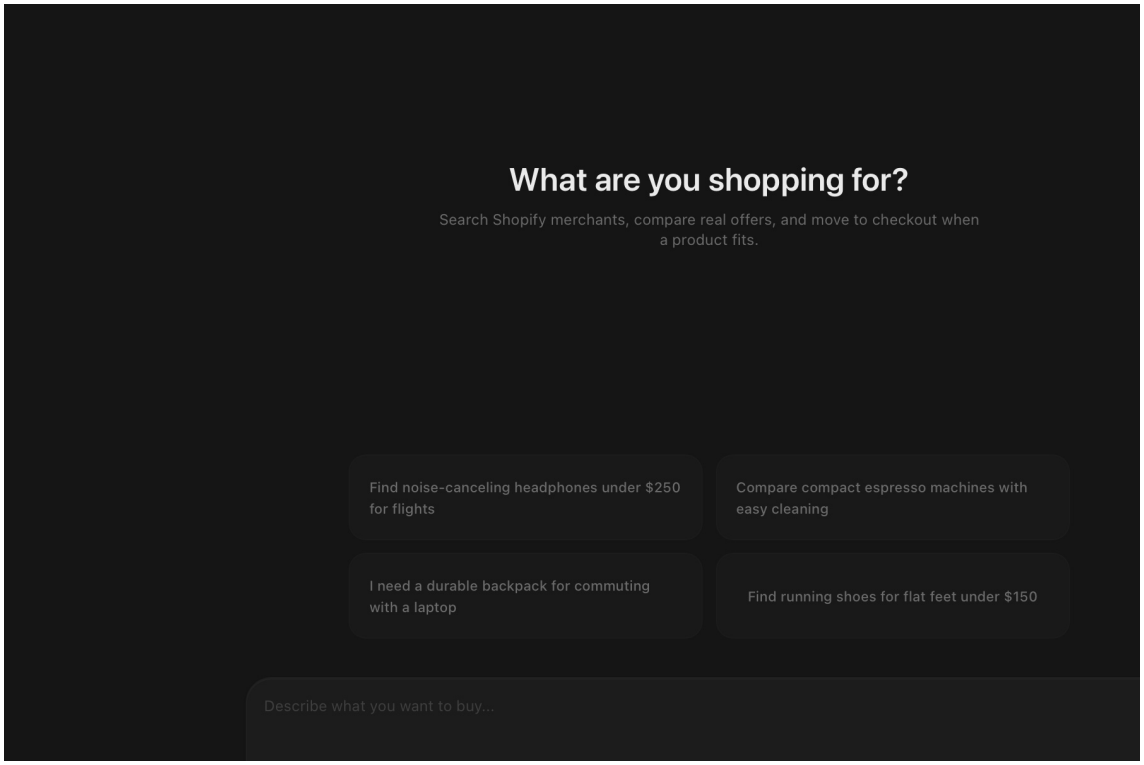

Figure 1.1: Shopping Agent chat interface

Chapter 2

Basic Concepts / Literature Review

The project combines agentic commerce, tool calling, generative user interfaces, streaming interaction, and persistent web application design. Shopify's UCP-compatible Catalog MCP exposes search_catalog, lookup_catalog, and get_product operations. These operations provide normalized products, variants, sellers, price ranges, media, specifications, availability, and merchant URLs that can be used as evidence for recommendations.

2.1 Agentic Commerce and Grounded Generative UI

Agentic commerce extends ordinary search by allowing a language model to choose and sequence tools according to a shopper's goal. In this project, Vercel AI SDK streamText manages multi-step model output and tool calls. Search results remain private working data until displayProducts emits a visible product grid. compareProducts, compareSellers, clarifyIntent, webSearch, and buyProduct each map to a dedicated interface component. This one-way contract makes tool output, rather than free-form prose, the source of visible product facts.

The frontend uses Next.js App Router, React, Tailwind CSS, shadcn/ui, SWR, and next-auth. The backend uses Node.js, strict TypeScript, Zod schemas, Vercel AI SDK, and a hand-written HTTP router. PostgreSQL stores users, chats, messages, and votes. Redis caches recommendation-feed and web-evidence data, while Shopify catalog search and catalog images deliberately bypass caching to preserve freshness and policy compliance. Playwright supplies browser automation for end-to-end tests and web-evidence retrieval.

Chapter 3

Problem Statement / Requirement Specifications

The problem is to help a buyer move from an imprecise natural-language request to a small, defensible set of purchasable products without hallucinated catalog facts. The solution must handle ambiguity, live cross-merchant search, follow-up refinement, comparison, seller selection, research evidence, streaming responses, persistence, and safe external links while remaining usable under free-tier model limits.

3.1 Project Planning

Development was planned around a vertical shopping journey: establish the Shopify catalog integration; define a track-agnostic agent registry; implement search, refinement, comparison, seller and checkout tools; render tool outputs as native components; add authentication and persistence; build the discovery surface; harden failure handling; and verify the system with unit, end-to-end, smoke, and scenario tests. Historical CSV and single-store approaches were retained only as reference after the runtime pivoted to the cross-merchant Catalog MCP.

3.2 Requirements Analysis (SRS)

Functional requirements include guest or registered access, creation and persistence of chats, natural-language product search, one-question clarification, search refinement, pagination, product detail retrieval, product and seller comparison, evidence lookup, structured result rendering, voting, and merchant-link handoff. Non-functional requirements include factual grounding, responsive design, low-friction demo access, deterministic behavior within a chat, acceptable recovery from dropped streams, secret isolation, URL validation, strict typing, and compatibility with local Docker services.

3.3 System Design

3.3.1 Design Constraints

The implementation requires Node.js 20 or later, a modern browser, Docker, PostgreSQL 16, Redis 7, frontend and backend environment files, and outbound HTTPS access to Shopify Catalog MCP and the selected model provider. The free-model strategy introduces rate-limit variability. Merchant catalog fields are incomplete in some records, currency values are not normalized in the interface, and web search depends on third-party page markup. These constraints are handled through explicit fallbacks and visible limitations rather than fabricated data.

3.3.2 System Architecture

The browser renders the React chat and discovery interfaces. Next.js owns routing, authentication-facing pages, server components, and client state. Requests are forwarded to the Node backend, where the active agent is built per chat with an isolated search memo and deterministic seed. The backend streams responses, executes tools, persists messages to PostgreSQL, uses Redis for selected caches, calls Shopify Catalog MCP for product data, and calls OpenRouter or DeepSeek for model inference. An internal secret protects the frontend-to-backend boundary.

Chapter 4

Implementation

Implementation follows a tool-oriented architecture. Each shopping capability has a typed input schema and a narrow responsibility, while the frontend maps completed tool parts to specific components. The current Track 1 agent exposes eleven tools: searchProducts, displayProducts, showMore, refineSearch, getProduct, compareProducts, compareSellers, buyProduct, clarifyIntent, webSearch, and webFetch. Tracks 2-5 retain compatible registry stubs for later work.

4.1 Methodology

A request is classified by intent in the system prompt. Broad requests may trigger one clarification; constrained or specific requests go directly to catalog search. Search state is reconstructed from saved message parts into a per-request SearchMemo. Relevance results keep the top two stable and deterministically shuffle the long tail using a chat-derived seed. A low-rating filter excludes products below 3.0 when enough stronger results exist. Comparisons look up two to four products, normalize seller and specification fields, and expose the cheapest available merchant link.

4.2 Testing and Verification Plan

Verification combines deterministic checks with live-agent scenarios. Vitest covers backend routing, authentication enforcement, history, messages, votes, documents, uploads, and server actions. TypeScript noEmit checks compile-time contracts. Playwright specifications cover chat input, API integration, authentication screens, suggested actions, stopping generation, and model selection. The self-test harness exercises eight shopping scenarios and records tool order, response text, failures, and rate limits in archived transcripts.

Test	Condition	System behavior	Expected result
T01 Auth guard	No user headers	Reject request	HTTP 401
T02 Search flow	Product + constraint	Catalog tools	Product cards
T03 Compare flow	Two product IDs	Compare tool	UI table

4.3 Result Analysis

The implemented system presents a dark, responsive chat interface with guest access, conversation history, rotating suggestions, an in-chat model selector, and a link to the discovery surface. Current backend verification passes all eleven Vitest cases and the strict TypeScript check. Archived scenario runs demonstrate successful catalog search, seller comparison, and product comparison sequences, while also documenting free-model rate limits and cases where the model emitted a clarification payload as text. Defensive frontend parsing converts these malformed outputs into usable interface states.

4.4 Quality Assurance

Quality assurance is enforced through grounded UI, schema validation, source-of-truth synchronization after streams finish, absolute HTTP/HTTPS link validation, hidden empty comparison rows, no-store catalog calls, and explicit refusal to answer compatibility questions without evidence. Errors are surfaced without inventing results; interrupted conversations can auto-resume; a 90-second client timeout provides recovery guidance; and malformed model prose is stripped of JSON envelopes, procedural narration, and duplicate markdown tables.

Chapter 5

Standards Adopted

5.1 Design Standards

The interface follows responsive web design, consistent component composition, keyboard-accessible Radix primitives, dark-mode theming, and progressive disclosure. Product facts are shown in cards and comparison matrices, while assistant prose remains short. The system separates browser, Next.js, backend, storage, catalog, and provider responsibilities. API boundaries use typed JSON, Zod validation, and explicit message-part states.

5.2 Coding Standards

The project follows strict TypeScript and repository-level formatting, naming, validation, and security conventions:

• Use strict TypeScript types and Zod schemas at tool and request boundaries.

• Use descriptive camelCase functions, PascalCase components, and domain-specific types.

• Keep each shopping tool focused on one operation and reuse normalized catalog models.

• Keep credentials in ignored environment files and protect backend calls with an internal secret.

• Run Biome or TypeScript checks and remove debug output before delivery.

• Use no-store requests for live catalog data and validate every external merchant URL before rendering.

5.3 Testing Standards

Testing follows a layered strategy: unit and route behavior with Vitest, static correctness with TypeScript, browser workflows with Playwright, deterministic smoke steps for the principal shopping journey, and scenario-based agent evaluation focused on tool sequencing and hallucination indicators. Failures and provider limits are preserved in transcript artifacts instead of being hidden or reclassified as passes.

Chapter 6

Conclusion and Future Scope

6.1 Conclusion

AI Shopping Agent demonstrates a practical architecture for conversational commerce grounded in live merchant data. It reduces search friction by translating natural-language intent into a sequence of clarification, catalog search, refinement, comparison, seller analysis, evidence lookup, and purchase handoff operations.

The strongest design result is the generative UI contract: product cards and comparison surfaces can appear only from completed tool outputs. Combined with strict schemas, persistence synchronization, link validation, and defensive rendering, this makes the system more trustworthy than a prose-only shopping chatbot. Current backend tests and type checks pass, while archived agent evaluations expose provider instability honestly.

6.2 Future Scope

Future work should add multi-currency normalization, saved carts and cross-chat preferences, richer accessibility testing, automated provider failover, merchant-specific catalog support, and stronger observability for tool latency and failure rates. The Track 2-5 registry stubs can be implemented for checkout recovery, checkout copilot, support, and merchant optimization without changing the core routing shape.

Further evaluation should use a stable paid model baseline, a larger catalog-quality benchmark, automated link-health checks, and human scoring for relevance, trust, time-to-recommendation, and purchase confidence. Native mobile clients and image-based similar-product search remain outside the present scope.

References

[1] Shopify, "Storefront MCP server and UCP catalog tools," Shopify Developer Documentation, 2026. Available: https://shopify.dev/docs/apps/build/storefront-mcp/servers/storefront

[2] Vercel, "AI SDK Core: streamText," AI SDK Documentation, 2026. Available: https://ai-sdk.dev/docs/reference/ai-sdk-core/stream-text

[3] Vercel, "Next.js App Router," Next.js Documentation, 2026. Available: https://nextjs.org/docs/app

[4] PostgreSQL Global Development Group, "PostgreSQL 16 Documentation," 2026. Available: https://www.postgresql.org/docs/16/

[5] Redis, "Redis Documentation," 2026. Available: https://redis.io/docs/latest/

[6] OpenRouter, "Quickstart Guide," OpenRouter Documentation, 2026. Available: https://openrouter.ai/docs/quickstart

[7] Microsoft, "Playwright Test: Introduction," Playwright Documentation, 2026. Available: https://playwright.dev/docs/intro

INDIVIDUAL CONTRIBUTION REPORT

AI SHOPPING AGENT

Ridhi Kumari
23052632

Abstract: AI Shopping Agent helps buyers discover, compare, and purchase products through a grounded conversational interface. The system uses Shopify Catalog MCP, typed agent tools, streaming model responses, and native product components so visible product facts originate from tool results. The implementation includes guest access, persistence, comparison and seller workflows, a discovery surface, error recovery, and layered verification.

Contribution scope: The work focused on the buyer-facing application, interaction design, frontend state and persistence synchronization, structured shopping components, defensive response parsing, authentication experience, discovery surface, testing support, screenshots, and project documentation.

Individual contribution and findings: Ridhi Kumari designed and implemented the frontend chat shell, message flow, multimodal composer, sidebar history, model selector, guest-first authentication pages, responsive styling, and the discovery experience. She developed the product grid, product detail, comparison table, seller comparison, evidence panel, option chips, and purchase call-to-action components. She also implemented the useChat transport wrapper, database refresh after stream completion, auto-resume integration, timeout recovery, and defensive parsing that unwraps JSON response envelopes, removes duplicate markdown tables and narration, and salvages clarification menus. The work demonstrated that grounding visible product UI in typed tool results provides a stronger reliability boundary than relying on model prose alone.

Individual contribution to project report preparation: Ridhi Kumari prepared the product description, interface and workflow sections, implementation summary, testing evidence, screenshots, decision rationale, standards, conclusion, and final formatting of this report.

Individual contribution for project presentation and demonstration: Ridhi Kumari prepared the walkthrough and demonstrated guest entry, constrained product search, clarification chips, product cards, comparison, seller selection, model switching, and checkout handoff.

Full Signature of Supervisor:
…….

Full signature of the student:
……..

TURNITIN PLAGIARISM REPORT

This report is mandatory for all projects and plagiarism must be below 25%.

Attach the official Turnitin plagiarism report before final submission.